

[< Go to the original](#)

Solving LLM's creativity paradox—AGI's next frontier

Where is LLM's creativity?



JayL

Follow



Towards AI a11y-light ~14 min read · September 5, 2025 (Updated: September 6, 2025) · **Free: Yes**

 Quantum Graviton Foam Theory (QGF Theory)

A unified model that treats spacetime and particles as emergent phenomena from a dynamic, quantized information field.

Response by ChatGPT

The result looks impressive at first glance; it builds upon existing ideas of quantum foam and applies it in a novel way, much like a physicist would. It even followed up with some potential experiments to run.

Experiment	QGF Signature	Feasibility	Current Status
Gravity-Induced Entanglement	Entanglement from gravity alone	Near-term	Under active experimental design
Black Hole Entropy Corrections	Logarithmic entropy deviations	Long-term	Observationally inaccessible
Minimal Length Scale Effects	High-energy deviation/noise	Medium-term	Constraints exist, no detection

Response by ChatGPT

For every significant scientific challenge I threw at Claude, ChatGPT, Grok, or Gemini. The state-of-the-art (SOTA) LLMs all responded with seemingly

This raises the question: Why aren't there groups of LLM scientists running around revolutionizing science?

The reason: **LLMs struggle with iterative creativity.**

The scientific method is based upon a constant cycle of working, testing, learning, and revising. A hypothesis may take years to mature into a theory, and micro-revisions happen almost daily.

LLMs lack this fundamental understanding of the trial-and-error cycle, which is innate to how humans create new knowledge.

In this article, I will examine the drives behind human creativity, define what it means to be creative, explain where LLMs fall short, and introduce a new state-of-the-art approach to address the creativity challenge.

To start, let's take a step back from AI and examine raw human creativity.

How we learned to create

anything because there is nothing to remember.

We are born with both a small **knowledge space** — the things we know and understand, and a small **action space** — the actions we can perform to solve problems. But in return, we are endowed with an incredibly elastic brain for learning (neuroplasticity).

There are two generally accepted learning paradigms in both children and machine learning:

- **Imitation learning** — watching a parent or reading a book. You trust that the source is likely correct and copy their knowledge or action. (supervised and unsupervised machine learning)
- **Trial and error** — discovering the way to the park by taking many detours, learning from our mistakes, updating our internal guide, and eventually finding the right path. (reinforcement machine learning)



Image by Gemini

As kids, we act stupidly, get hurt, learn from our mistakes, and try something else. This core loop of **trial and error** is an innate way for us to synthesize new knowledge. This solidifies the link between knowing and doing, expanding both the knowledge and action spaces.

But as we grow up, we are sent to school and switch primarily to **imitation learning**. The reason is that imitation saves time. Because we trust the teachers and textbooks, we can skip the trial-and-error exploration and go straight to the result.

knowledge from one person to another, regardless of time, is what makes us special.

Once one of us discovers something, we all benefit indefinitely.

But there is a problem with this method.

While imitation learning is efficient and robust for transferring knowledge, it is **not a path for discovering new knowledge**.

What if we want to visit a park that is not on any map? We can search on Google maps, ask teachers or friends, but if nobody knows how to get there, then there is nothing to imitate.

We are left with only trial and error.

There are many ways to conduct trial-and-error; we can go in a straight line, head toward the nearest greenery, or follow street signs leading to places that are likely to have a park. There is more than one method to finding the way to the park, but some take longer and others may take forever. So people naturally came up with a way to judge the effectiveness of our methods— **creativity**.

The creativity measurement

Fundamentally, I define creativity as **the ability to generate original, quality knowledge**.

Most popular definitions of creativity revolve around the idea of originality, and this is correct but incomplete. Many ideas are original, but many original ideas are also bad. Being original is not enough; you also have to generate value.

Many people write songs, but creative artists write hit songs, and only the best artists create masterpieces that change people's lives.

Of course, this means that creativity is often a subjective measurement. My favorite band may not be yours, and each person values things differently. However, as long as an original idea can generate enough value, it can be considered creative.

To summarize, I believe two things define an entity's creativity in an area:

1. The coverage and robustness of the knowledge base.
2. The efficiency of the action space in synthesizing new knowledge.

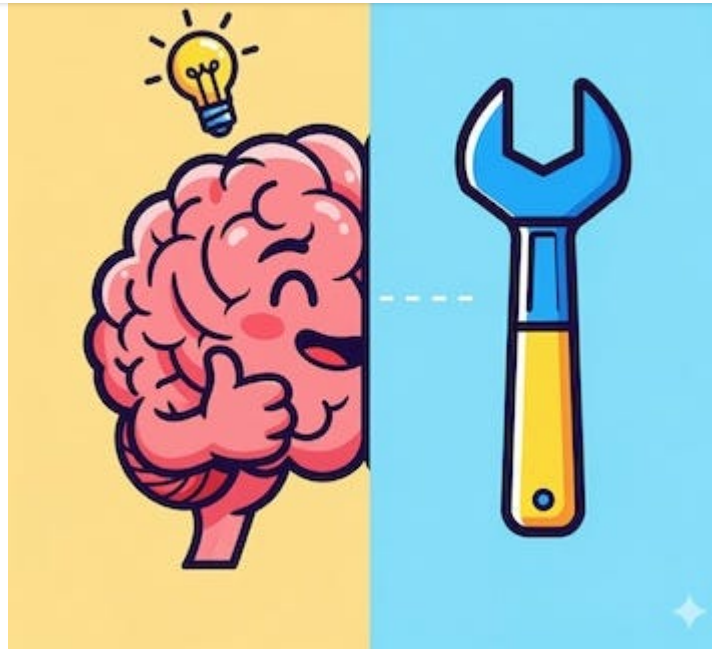


Image by Gemini

Great musicians often possess musical knowledge far beyond that of the average person, and their ability to combine, modify, and channel past sounds into new music is extraordinary.

While creativity is subjective in artistic expressions, it becomes objective in scientific exploration. If a hypothesis failed to solve the problem or fit the observation, then it is not a good one, and no one will debate you otherwise.

Next, let's take a closer look at how LLMs understand the world.

The LLM Paradox: Superior Knowledge, Limited Action

Fundamentally, all **Deep Learning** models work by trying to retain a large amount of information (training data) within limited internal dimensions (neuron weights), which forces the model to compress facts into concepts. LLMs are no different, but they do have an ace up their sleeve — the **transformer**.

The transformer features an **attention mechanism to identify relationships between different parts of the training data**. It scales so efficiently that, for the first time, we can use nearly the entire internet to train models.

I won't delve into too much technical detail of the transformer here, but here is a high-level summary of how a **base model** (no fine-tuning) of an LLM is trained.

1. The LLM takes in some sequence of data (**tokens**).
2. It passes the tokens one by one through its "attention" mechanism (**Key, Query, Value** tensors), which then learns some pattern between the current and past tokens.

4. The loss function tracks the error between the predicted token and the actual token, reporting it back to the LLM.
5. The LLM then adjusts its internal parameters (weights) to lower the error (**Backpropagation**).
6. Repeat steps 1–4 until the overall error is sufficiently low or the training data is exhausted.

I'm simplifying many essential concepts here. If you are interested in learning the technical side of transformers, I highly recommend the transformer video series by [3blue1brown](#).

ChatGPT, Gemini, Claude, or Grok may have different architectures in how they organize their weights, but they all follow the same training cycle.

We refer to this stage as LLM's **imitation learning**, where the model attempts to transfer existing knowledge into internal representations. As a result of training on nearly the entire internet, modern LLMs end up with a knowledge base that far surpasses that of any human.

parameters is an active area of research known as mechanistic interpretability.

However, there is a major pitfall in this training approach that people rarely talk about:

LLMs are trained on results, not methods.



Image by Gemini

When most people publish things on the internet, they publish the result, not the years of work it took.

author's head. Now consider the actual book that was published, which may tell a great story, but says little about how the author wrote it.

Even for academic papers where methods and experiments are clearly documented, it's not possible to record the hundreds of daily mental hurdles that a scientist has to overcome in a twenty-page paper.

Nor does the average person give detailed reasons for answering a question on Reddit. (Which is heavily referenced by SOTA LLMs)

Since an overwhelming amount of training data focuses on results rather than methods, LLMs mirror this distribution by possessing a staggering amount of factual knowledge but lacking information on how the facts were discovered.

The result is devastating.

The consequence of result-based training

When we ask a pretrained LLM to calculate 2×4 , it doesn't pull up a calculator or perform mental arithmetic like we do. Instead, it searches through its knowledge

This is precisely why LLMs sometimes fail at simple tasks, such as counting words. Because it was **trained on tokens (different from words)** and lacks any counting logic, it will search for the most likely internal relationship instead of actually counting.

State-of-the-art models often address these problems with tool use- fixing the logic by grounding the LLM with external tools (Python programming). This is a good temporary solution to patch over known logic flaws, but it runs into problems when LLMs encounter logic problems beyond the tool's capability.

Creativity depends on both the knowledge and action space. LLM has an incredible knowledge space, but its action space contains only one action: **next token prediction**.

And token prediction is a **single-pass, shallow thought process**.

Humans have different thought processes that run in parallel when we think, some slow and methodical, others are fast and decisive. The different thinking

Token prediction is similar in function to the brain's **cortical columns**. Given past stimuli, it attempts to predict the next stimuli.

Sequence prediction is a fundamental building block of thought, but cortical columns can't form complex thoughts on their own, and neither can the LLM.

If you would like to learn more about cortical columns and the brain, I highly recommend Jeff Hawkins' book: [A Thousand Brains](#).



Image by Gemini

In summary, a base LLM's inability to form complex thoughts, combined with deep logical flaws, means it cannot use trial-and-error methods.

To begin finding a solution, let's examine how the base model is fine-tuned through reinforcement learning.

Reinforcement Learning through human feedback.

data.

To ensure the model outputs tokens that are harmless, helpful, and aligned with human preferences, **reinforcement learning through human feedback (RLHF)** is used. The training cycle is as follows.

Prep:

1. Get a custom dataset of human preferences. This can take the form of a Q&A that includes both preferred and non-preferred responses.
2. A reward model is trained on the custom dataset to judge the quality of the LLM's answers.

Training:

1. The base LLM generates a response to a question.
2. The reward model gives an alignment score of the response.
3. The base LLM updates its weights to generate more aligned responses.
4. Repeat steps 1–3 until an average alignment threshold is reached.

deeper explanation of RLHF, check out [Hugging Face's Deep RL course page](#).

In this way, a base model can learn from its own actions and output tokens that are helpful to humans without directly imitating the preferences data.

This is a promising way to incorporate the trial-and-error process into the LLM's training; however, it too suffers from a critical shortcoming: **the learning stops as soon as the RLHF process ends.**

The model may have learned helpful alignment through an externally set up trial-and-error process, but the LLM has not learned how to perform trial and error for anything else.

To echo an old proverb, we gave the LLM more fish instead of teaching it how to fish.

I have discussed many shortcomings of LLMs so far, and the road to an LLM scientist is fraught with peril.

However, this does not mean that true creativity is impossible to achieve.

Beyond Plan A: The Hidden Originality in LLMs

A recent Anthropic podcast referred to LLM responses as forming different "plans". I found this analogy to be spot on in describing the statistical nature of LLMs.

When we ask ChatGPT or Claude a question for the first time, it will most likely answer with Plan A — the statistically probable result, fine-tuned to be as "correct" as possible. This means Plan A favors popular things that have been proven to be effective.

Plan B has a slightly lower probability of being correct and proven, followed by Plan C, which has even less, and so on.

The likelihood of an LLM choosing a particular plan often corresponds with "**temperature**" — the randomness of the token selection process. The higher the temperature, the more random the answers because less likely tokens are selected more often.

When I asked Claude Sonnet to generate a Python script of a biological neuron, its plan A was to create an artificial "**point neuron**". That is how people have traditionally represented neurons in deep learning. Claude had difficulty

However, this internal bias of LLMs doesn't mean it cannot go beyond the status quo; it's just hidden away.

Suppose we can force the LLM to abandon Plans A through D and proceed to Plan E. In that case, it has a very low chance of correctly answering the question because although it has found some patterns in its training data, they weren't popular/aligned enough to become accepted solutions. So Plan E+ has extremely high chances of failure, but that chance is not 100%.

Because Plan E+ has found some statistically unlikely, but existing correlations, there are two likely scenarios:

- Someone tried the idea and failed, so nobody really adopted it (99% of the time)
- It's an original idea that nobody had tried before (1%)

For example, if you asked which animals can fly and rejected every bird answer (Plan A to D), then a high-temperature LLM might suggest a fish.

If you ask a child the same question and they answer with 'flying fish,' you might even say that they were being creative.

To summarize, while original ideas can be found in the attention mechanisms, three key shortcomings stop the LLM's creative process.

1. Training on results, not methods.
2. Shallow token prediction thinking.
3. Finetuned towards correctness.

How can we address these fundamental issues and encourage LLMs to favor creativity?

Google's DeepMind has an interesting approach.

I'm going to skip over "chain of thought" and other prompting techniques here, as they gave LLMs more token passes to "think" in a loop. However, it is still trying to get to a correct answer rather than promoting original ideas.

A Path Forward: Alpha Evolve and Beyond

When AlphaGo finally defeated Lee Sedol in the game of GO, it didn't do it from imitation alone. AlphaGo played thousands of games against itself in a trial-and-error loop, which gave rise to the famous move 37 that shook the world. It was a move that no human player would have made, yet it directly led to AlphaGo's victory.

And it did this without even having a transformer.

The critical thing to note here is that AlphaGo not only used a deep neural network, but also an external search algorithm — **monty carlo tree search**.

This combination enabled AlphaGo to exhibit the first universally acknowledged display of creativity by an AI. Move 37 was both original and valuable in winning the game.

And now, DeepMind has taken this approach one step further and applied it to an LLM, with **Alpha Evolve**.

Just like AlphaGo, Alpha Evolve uses an external search algorithm on top of LLMs to generate new search spaces. In this case, it's the **evolutionary algorithm** that

The basic AlphaEvolve flowchart looks like this:

1. Initialization — the user provides the initial solution along with a method to evaluate it.
2. Prompt sampling—the sampler takes the programs from the program database to construct new prompts.
3. LLM revision — the ensemble of LLMs takes in the prompt and generates new solutions.
4. Evaluation — the evaluator scores the new solutions, and only promising solutions are registered back to the program database.
5. Continue steps 2–4 until the desired evaluation score is achieved.

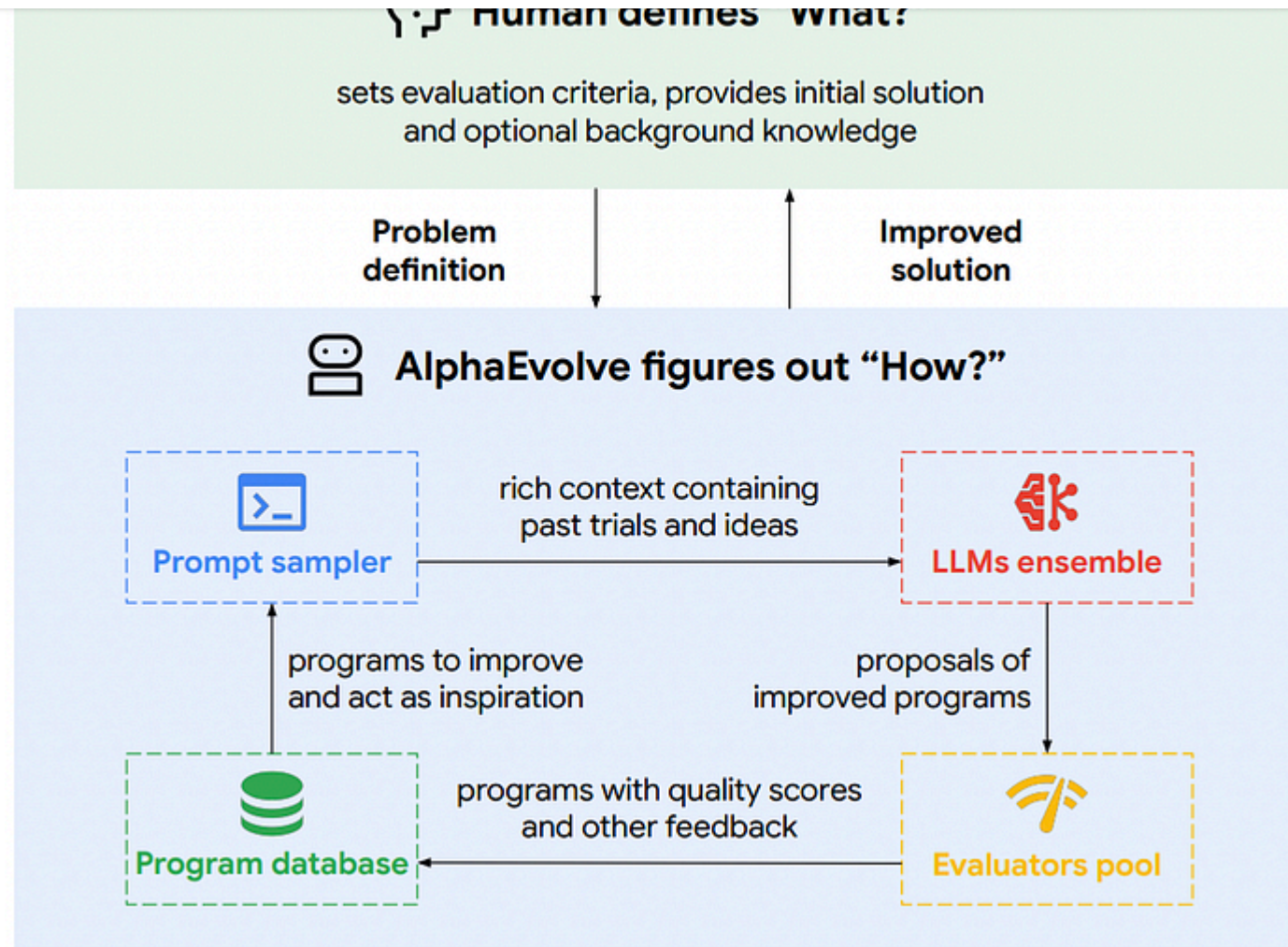


Figure from the AlphaEvolve whitepaper: [Link](#)

Let's take a look at how AlphaEvolve addresses the three major blockers we listed earlier:

- The evolutionary search algorithm is a trial-and-error method, and unlike RLHF, it directly gives LLMs the ability to perform trial-and-error learning for a variety of problems, not just alignment.

2- Shallow token prediction thinking.

- The ensemble methods combine fast thinking, breath-first search LLMs with SOTA depth-first LLMs to simulate a diversity of low-level thinking.
- The evolutionary search algorithm simulates a high-level thinking process that utilizes lower-level LLM outputs to plan future strategies.
- The program database, serving as the long-term memory, tracks past knowledge and failures for the whole system.

3- Finetuned towards correctness.

- The evolutionary search algorithm promotes exploration and offers a different evaluation metric for value generation rather than relying on the finetuned alignment.

The key insight behind AlphaEvolve is not to think of LLMs as a complete standalone thinker, but rather as a component of a larger reasoning framework.

optimizing Google's existing systems.

This proves that there is untapped creativity within LLMs, and with the proper framework, we can unlock their true creative potential.

I won't delve into Alpha Evolve's technical details here, as I will do a tutorial on how to build an AlphaEvolve clone from scratch in Python next!

Stay tuned!

The future

The holy grail of AI has always been scientific discovery.

Imagine if we could clone the world's top scientists to work around the clock on all of humanity's most challenging problems.

The rate of progress would be unprecedented in history, and we may even reach the stars in our lifetime.

Please comment below if you found this interesting, want to see an in-depth technical tutorial, or if there is anything you think I've missed. I will be happy to answer any questions!

If you'd like to connect, you can find me on [LinkedIn](#).

Thanks for reading!

[#ai](#) [#llm](#) [#machine-learning](#) [#deep-learning](#) [#data-science](#)